Reporte Técnico de Actividades Práctico-Experimentales Nro. 001

Construcción y simulación computacional de un modelo matemático

1. Datos de Identificación del Estudiante y la Práctica

Nombre del estudiante(s)	Emilio Josue Flores Gallardo
Asignatura	Simulación
Ciclo	5
Unidad	U1: Introducción a la Simulación
Resultado de aprendizaje de la unidad	R1. Aplica análisis estadísticos de los datos de la Simulación para mejorar la calidad de los resultados, bajo los principios de solidaridad, transparencia, responsabilidad y honestidad.
Práctica Nro.	001
Título de la Práctica	Construcción y simulación computacional de un modelo matemático
Nombre del Docente	José O. Guamán Q.
Fecha	01 y 02 de octubre de 2026
Horario	Jueves 10h30 a 11h30 / Viernes 07h30 a 09h30
Lugar	Aula
Tiempo planificado en el Sílabo	3 horas
Repositorio Git	Repositorio GitHub

2. Objetivo(s) de la Práctica

Objetivo general: comprender el proceso de construcción de un modelo matemático y analizar su comportamiento mediante Python.

Objetivos específicos:

- Identificar variables, parámetros y relaciones matemáticas de un sistema.
- Implementar un modelo matemático en Python.
- Analizar gráficamente la influencia de los parámetros.

3. Materiales, Reactivos, Equipos y Herramientas

Software y librerías:

- Python 3.12 con entorno virtual (`venv`) y librerías listadas en `requirements.txt`.
- Librería **Matplotlib** para mostrar la tabla de resultados y graficar la curva del índice de lluvia.

- Librería **NumPy** para el soporte numérico del proyecto.
- Visual Studio Code como editor de código, junto con Git y GitHub para el control de versiones.

Equipos: computador con procesador de 2 núcleos o superior, mínimo 4 GB de RAM y sistema operativo Linux, macOS o Windows.

4. Procedimiento / Metodología Ejecutada

4.1. Fundamento teórico

Un modelo matemático representa un fenómeno real usando variables, parámetros y fórmulas. En esta práctica se simula el clima durante el día para saber en qué horas hay probabilidad de lluvia mediante la ecuación:

$$I = w_h H + w_n N + w_{tf} T_f \tag{1}$$

Los pesos deben sumar $w_h + w_n + w_{tf} = 1{,}0$, siendo los valores base $w_h = 0{,}50$, $w_n = 0{,}30$ y $w_{tf} = 0{,}20$. Las variables de entrada son:

- H: humedad normalizada entre $[0, 1]$, calculada como $H = \text{Humedad}/100$.
- N: nubosidad normalizada entre $[0, 1]$, calculada como $N = \text{Nubosidad}/100$.
- T_f: factor de temperatura asignado según la Tabla 4.1.

Tabla 1. Tabla de asignación del factor de temperatura (T_f).

Temperatura (°C)	**Factor T_f**	**Temperatura (°C)**	**Factor T_f**
Menor o igual a $10\,°$C	1.00	$20\,°$C	0.50
$12\,°$C	0.90	$22\,°$C	0.40
$14\,°$C	0.80	$24\,°$C	0.30
$16\,°$C	0.70	$26\,°$C	0.20
$18\,°$C	0.60	Mayor o igual a $28\,°$C	0.10

Con el índice I calculado, el programa determina el estado del clima usando las reglas de la Tabla 4.1.

Tabla 2. Reglas de clasificación según el índice I.

Intervalo del Índice (I)	**Estado Pronosticado**
$I < 0{,}40$	Sin lluvia
$0{,}40 \leq I < 0{,}60$	Baja posibilidad
$0{,}60 \leq I < 0{,}75$	Lluvia probable
$I \geq 0{,}75$	Lluvia

Nota sobre las temperaturas impares: la tabla de la guía incluye mediciones desde las 06:00 hasta las 22:00, pero las horas 20:00 ($17\,°$C) y 22:00 ($15\,°$C) tienen temperaturas impares que no aparecen en la Tabla 4.1. En el código solo se usaron las temperaturas pares tabuladas; cuando el método `calcular_tf` recibe un valor impar no registrado, devuelve `None` y el controlador omite esa fila.

4.2. Las 7 etapas de un proyecto de simulación

El trabajo se desarrolló siguiendo las 7 etapas de un proyecto de simulación (Tabla 4.2):

1. **Definición del problema (Identificar el problema y objetivos):** Se identifica qué problema se quiere estudiar o resolver y cuáles son los objetivos de la simulación. En este caso, se busca predecir en qué horas del día lloverá a partir de datos del clima y ver cómo cambia el resultado al ajustar los pesos.
2. **Formulación conceptual (Cómo funciona el sistema real):** Se determina cómo funciona el sistema en la realidad: elementos, variables, relaciones, reglas y procesos que se van a representar. Se definieron las variables H, N y T_f, la fórmula $I = w_h H + w_n N + w_{tf} T_f$ y los cuatro rangos de estado.
3. **Recolección y análisis de datos (Obtener datos necesarios):** Se obtienen los datos necesarios del sistema real, como tiempos, cantidades, frecuencias y probabilidades. Se tomaron los registros horarios de la guía (06:00 a 22:00) y se filtraron las horas con temperaturas impares ($17\,°$C y $15\,°$C) por no estar en la tabla de factores.
4. **Construcción del modelo (Transformar el modelo conceptual):** Se transforma el modelo conceptual en un modelo de simulación utilizando herramientas o software especializado. Se programó el modelo en Python usando arquitectura MVC y Matplotlib para mostrar la tabla y la gráfica.
5. **Verificación y validación (Comprobar el modelo):** *Verificación:* comprobar que el modelo fue construido correctamente en el código, revisando el cálculo con el ejemplo de la guía ($I = 0{,}81$) y validando que los tres pesos sumen $1{,}0$. *Validación:* comprobar que el modelo representa adecuadamente el comportamiento del sistema real, confirmando que en la tarde (mayor humedad y menor temperatura) el índice prediga lluvia.
6. **Experimentación y análisis de resultados (Ejecutar escenarios y analizar):** Se ejecutan diferentes escenarios o condiciones y se analizan los resultados obtenidos para responder a los objetivos planteados. Se probó el modelo con los pesos base $(0{,}50, 0{,}30, 0{,}20)$ y luego con pesos ajustados $(0{,}30, 0{,}40, 0{,}30)$ para comparar los cambios.
7. **Implementación y documentación (Conclusiones y recomendaciones):** Se presentan las conclusiones y recomendaciones, se documenta el modelo y, cuando corresponde, se aplican los resultados obtenidos en el sistema real. Se armó el repositorio con su `README.md`, `requirements.txt`, `.gitignore` y este informe técnico.

Tabla 3. Resumen de las 7 etapas aplicadas en la práctica.

N.	Etapa	Propósito	Aplicación en APE 001
1	Definición del problema	Identificar el problema a resolver y los objetivos.	Predecir las horas con posibilidad de lluvia y analizar el efecto de los pesos.
2	Formulación conceptual	Definir elementos, variables, relaciones y reglas del sistema.	Plantear la fórmula $I = w_h H + w_n N + w_{tf} T_f$, la tabla de T_f y las reglas de estado.
3	Recolección y análisis de datos	Obtener tiempos, cantidades y datos de entrada.	Cargar la tabla horaria de 06:00 a 22:00 y descartar temperaturas impares no tabuladas.
4	Construcción del modelo	Pasar el modelo conceptual a código en software.	Programación en Python con estructura MVC y gráficas en Matplotlib.
5	Verificación y validación	Comprobar que el código no tenga errores y refleje la realidad.	Prueba con el cálculo base ($I = 0{,}81$), control de suma de pesos igual a 1 y revisión de resultados.
6	Experimentación y análisis	Probar escenarios y analizar los resultados.	Ejecución comparativa entre el escenario base y el escenario con pesos modificados.
7	Implementación y documentación	Presentar conclusiones, recomendaciones y documentar.	Entrega de este reporte y subida del proyecto completo a GitHub.

4.3. Arquitectura MVC del código

El programa se dividió en tres paquetes siguiendo el patrón **Modelo–Vista–Controlador (MVC)** y manteniendo cada archivo con una responsabilidad clara (Tabla 4.3).

Tabla 4. Archivos y módulos del proyecto APE 001.

Archivo / Clase	Función en el programa
`model/model.py` (*ModeloClima*)	Guarda el diccionario `tabla_tf` con las temperaturas pares, normaliza humedad y nubosidad, calcula el índice I y devuelve el estado del clima.
`view/view.py` (*VistaClima*)	Genera con Matplotlib la ventana de la tabla de resultados y la ventana con la gráfica lineal del índice I.
`controller/controller.py` (*ControladorClima*)	Recorre los datos horarios, filtra los registros donde T_f es `None`, llama a los cálculos del modelo y envía la lista final a la vista.
`main.py`	Contiene la lista de datos de entrada, muestra el menú en consola y verifica que los nuevos pesos ingresados por el usuario sumen $1{,}0$.

4.4. Escenarios y pasos de ejecución

En la Tabla 4.4 se muestran las dos configuraciones de pesos usadas para evaluar el programa.

Tabla 5. Escenarios probados en la simulación.

Escenario	Descripción	Peso w_h	Peso w_n	Peso w_{tf}	Suma
Escenario 1	Parámetros base de la guía	0.50	0.30	0.20	1.00
Escenario 2	Ajuste dando mayor peso a nubosidad y temperatura	0.30	0.40	0.30	1.00

Pasos que ejecuta el programa:

1. Cargar en `main.py` los registros de hora, humedad, nubosidad y temperatura.
2. Mostrar en consola el menú con las opciones: *1. Usar parámetros base* y *2. Ingresar nuevos parámetros*.
3. Si se elige la opción 2, leer por teclado w_h, w_n y w_{tf}, cambiar comas por puntos si hace falta y comprobar que $|w_h + w_n + w_{tf} - 1{,}0| < 10^{-9}$.
4. Crear el objeto `ControladorClima` pasándole los datos y los pesos elegidos.
5. Recorrer cada hora y buscar T_f con el método `calcular_tf` del modelo, saltando las filas con temperaturas impares no incluidas en la tabla.
6. Calcular las variables normalizadas $H = \text{Humedad}/100$ y $N = \text{Nubosidad}/100$.
7. Obtener el índice $I = w_h H + w_n N + w_{tf} T_f$ y asignar su estado según los rangos.
8. Mostrar con `VistaClima` la tabla de datos calculados y la gráfica del índice I.

5. Resultados

5.1. Salida en consola

En la Figura 1 se muestra la captura de la consola al ejecutar `main.py` con las opciones del menú interactivo.



Figura 1: Ejecución del menú interactivo y validación de pesos en consola.

5.2. Escenario 1: Parámetros base

Al correr la opción 1 ($w_h = 0{,}50$, $w_n = 0{,}30$, $w_{tf} = 0{,}20$), se obtienen los resultados de la Tabla 5.2. Las horas 20:00 ($17\,^{\circ}$C) y 22:00 ($15\,^{\circ}$C) no aparecen porque sus temperaturas son impares.

Tabla 6. Resultados con parámetros base ($w_h = 0{,}50, w_n = 0{,}30, w_{tf} = 0{,}20$).

Hora	Humed.	Nubos.	Temp.	H	N	T_f	Índice I	Estado
06:00	65 %	40 %	14 °C	0.65	0.40	0.80	0.605	Lluvia probable
08:00	70 %	50 %	16 °C	0.70	0.50	0.70	0.640	Lluvia probable
10:00	68 %	45 %	18 °C	0.68	0.45	0.60	0.595	Baja posibilidad
12:00	60 %	30 %	22 °C	0.60	0.30	0.40	0.470	Baja posibilidad
14:00	75 %	70 %	20 °C	0.75	0.70	0.50	0.685	Lluvia probable
16:00	85 %	85 %	18 °C	0.85	0.85	0.60	0.800	Lluvia
18:00	92 %	95 %	16 °C	0.92	0.95	0.70	0.885	Lluvia

En las Figuras 5 y 5 vas las capturas de las ventanas de Matplotlib del Escenario 1.

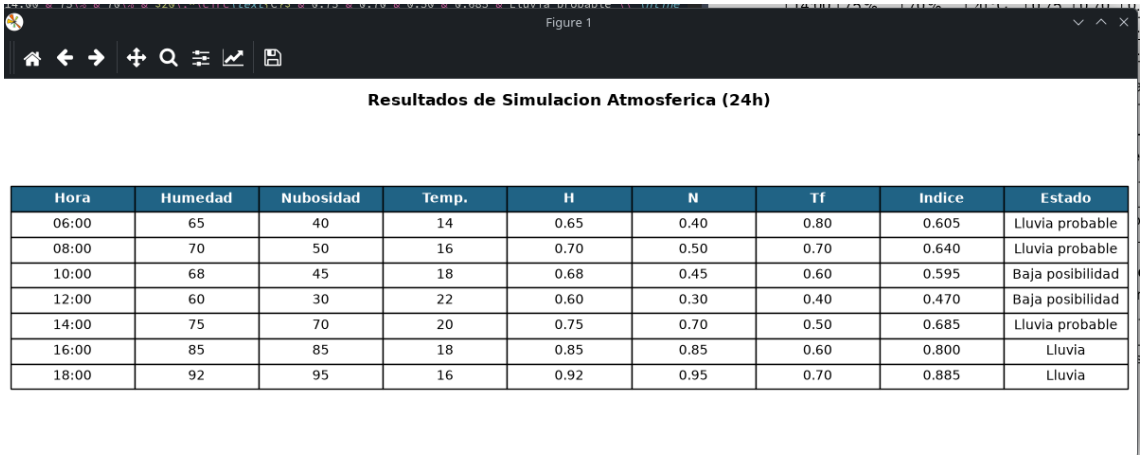


Figura 2: Tabla de resultados mostrada por Matplotlib con los pesos base.

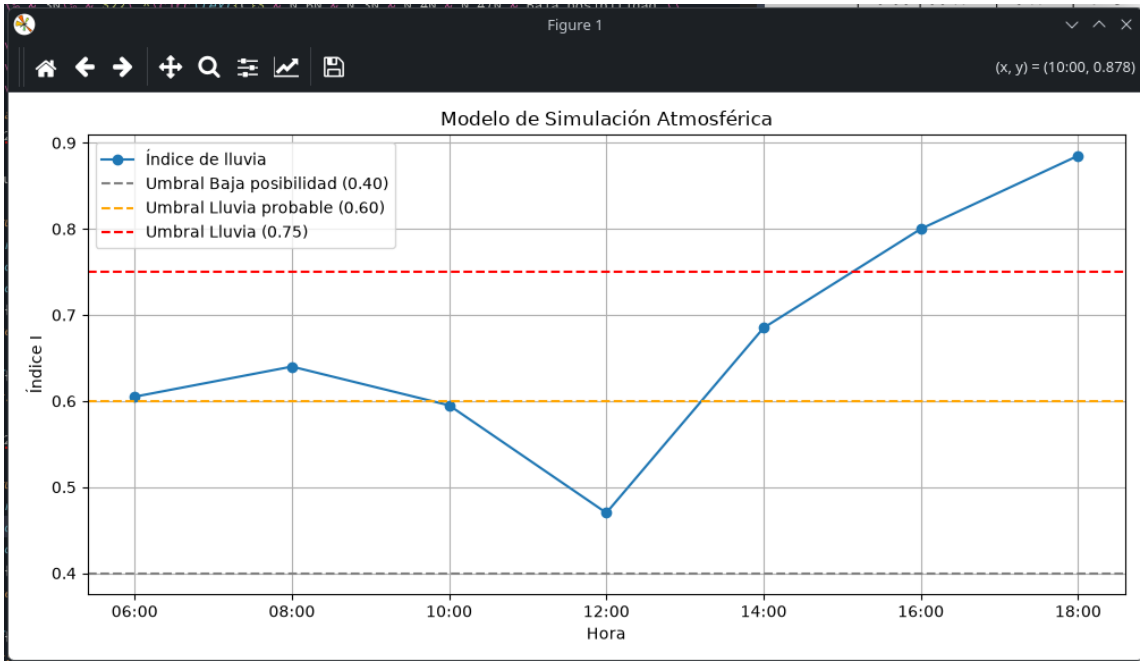


Figura 3: Gráfica mostrada por Matplotlib con los pesos base.

5.3. Escenario 2: Ajuste de parámetros

Al ingresar por consola los pesos ajustados $w_h = 0{,}30$, $w_n = 0{,}40$ y $w_{tf} = 0{,}30$, el programa arroja los valores de la Tabla 5.3.

Tabla 7. Resultados con parámetros ajustados ($w_h = 0{,}30, w_n = 0{,}40, w_{tf} = 0{,}30$).

Hora	Humed.	Nubos.	Temp.	H	N	T_f	Índice I	Estado
06:00	65 %	40 %	14 °C	0.65	0.40	0.80	0.595	Baja posibilidad
08:00	70 %	50 %	16 °C	0.70	0.50	0.70	0.620	Lluvia probable
10:00	68 %	45 %	18 °C	0.68	0.45	0.60	0.564	Baja posibilidad
12:00	60 %	30 %	22 °C	0.60	0.30	0.40	0.420	Baja posibilidad
14:00	75 %	70 %	20 °C	0.75	0.70	0.50	0.655	Lluvia probable
16:00	85 %	85 %	18 °C	0.85	0.85	0.60	0.775	Lluvia
18:00	92 %	95 %	16 °C	0.92	0.95	0.70	0.866	Lluvia

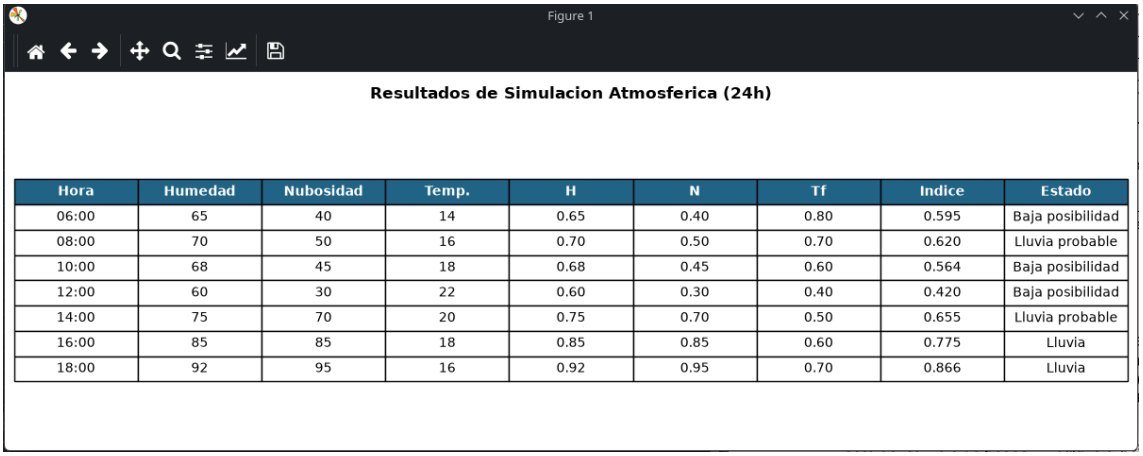


Figura 4: Tabla de resultados mostrada por Matplotlib con los pesos modificados.

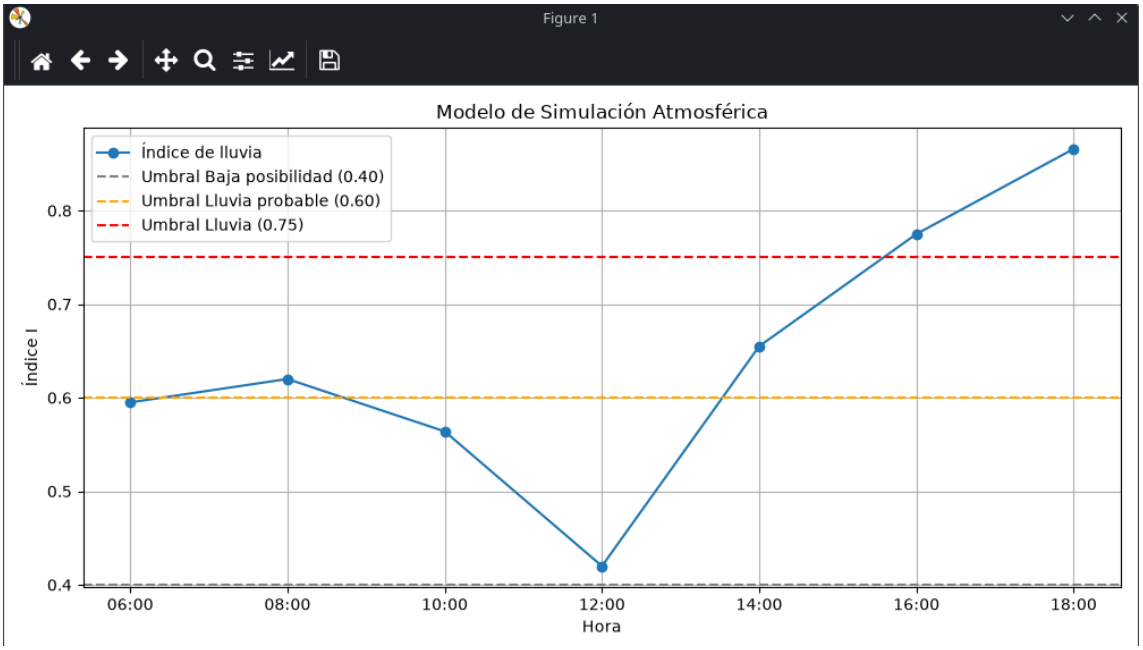


Figura 5: Gráfica mostrada por Matplotlib con los pesos modificados.

5.4. Análisis de resultados

- **Bajada al mediodía y subida en la tarde:** En los dos escenarios la gráfica baja desde las 08:00 hasta tocar su punto más bajo a las 12:00 ($I = 0{,}470$ en el base y $I = 0{,}420$ en el ajustado). Esto pasa porque al mediodía sube la temperatura a $22\,°$C ($T_f = 0{,}40$) y bajan tanto la humedad ($60\,\%$) como la nubosidad ($30\,\%$). Desde las 14:00 la curva vuelve a subir rápido y a las 16:00 y 18:00 pasa de $0{,}75$, entrando en estado de *Lluvia*.
- **Efecto del cambio de pesos:** Al bajarle el peso a la humedad de $0{,}50$ a $0{,}30$ y subirle a la nubosidad ($0{,}40$) y temperatura ($0{,}30$), todos los índices del día bajaron un poco porque en estos datos la humedad tenía valores altos. El cambio más claro se ve a las 06:00, donde el índice baja de $0{,}605$ a $0{,}595$ y pasa de *Lluvia probable* a *Baja posibilidad* por estar justo en el límite de $0{,}60$.
- **Sobre las temperaturas impares:** Como no se usaron las filas de las 20:00 ($17\,°$C) ni de las 22:00 ($15\,°$C), la gráfica tiene 7 puntos en lugar de 9.

5.5. Cumplimiento de resultados esperados

Tabla 8. Revisión de los resultados pedidos en la guía.

Resultado esperado	**Cumplimiento en la práctica**
Modelo implementado correctamente.	Código funcional en Python organizado en carpetas MVC (`model`, `view`, `controller`).
Gráficas de crecimiento y decrecimiento.	Gráfica en Matplotlib donde se ve la caída hacia las 12:00 y la subida hacia las 18:00.
Identificación de la influencia de los parámetros.	Comparación directa entre los pesos base y los nuevos pesos en las Tablas 5.2 y 5.3.
Interpretación matemática del comportamiento.	Explicación de cómo influyen H, N, T_f y los umbrales en el resultado final.

6. Preguntas de Control

1. **¿Qué es un modelo matemático?**
   Es una representación de un sistema o fenómeno real mediante fórmulas, variables y reglas matemáticas que sirve para entender cómo funciona y predecir su comportamiento sin tener que probar en el sistema físico [1, 2].
2. **¿Cuál es la diferencia entre variable y parámetro?**
   - **Variable:** Es un dato que va cambiando con el tiempo o en cada medición, como la humedad (H), la nubosidad (N), el factor de temperatura (T_f) y el índice resultante (I).
   - **Parámetro:** Es un valor fijo que se define para configurar el modelo antes de correrlo, como los pesos w_h, w_n y w_{tf} (o la tasa r vista en clase) [2, 3].
3. **¿Qué representa P_0?**
   En los modelos de crecimiento como $P(t) = P_0 e^{rt}$, P_0 es el valor o población inicial en el tiempo $t = 0$, es decir, el punto desde donde arranca la simulación [2].

4. **¿Qué ocurre cuando $r < 0$?**
   Cuando la tasa r es negativa, la cantidad disminuye con el tiempo en lugar de crecer (decrecimiento exponencial), acercándose cada vez más a cero [2].
5. **¿Qué limitaciones tiene el modelo exponencial?**
   - Supone que el crecimiento sigue para siempre sin límite de recursos, lo cual no pasa en la realidad cuando $r > 0$.
   - No toma en cuenta variaciones al azar ni cambios bruscos del entorno por ser totalmente determinístico [2, 3].

7. Conclusiones

- Se implementó el modelo climático en Python usando arquitectura MVC, logrando separar los cálculos matemáticos, el control de datos y las gráficas en Matplotlib de forma ordenada.
- Con las gráficas se comprobó cómo varía el índice I durante el día: baja al mediodía ($I = 0{,}470$) cuando sube el calor y hay pocas nubes, y sube en la tarde hasta marcar *Lluvia* a las 16:00 ($I = 0{,}800$) y 18:00 ($I = 0{,}885$).
- Al cambiar los pesos en el segundo escenario se vio que una pequeña variación en los parámetros puede cambiar el pronóstico cuando el índice está cerca de un límite, como pasó a las 06:00 al bajar de $0{,}605$ a $0{,}595$.

8. Bibliografía / Referencias

1. J. O. Guamán Q., *Guía de Actividades Práctico-Experimentales Nro. 001: Construcción y simulación computacional de un modelo matemático*. Carrera de Computación, FEIRNNR, Universidad Nacional de Loja, 2026.
2. Asignatura Simulación, *Diapositivas de la Semana 1: Introducción a la Simulación y Modelos Matemáticos*. Carrera de Computación, FEIRNNR - UNL, 2026.
3. A. M. Law, *Simulation Modeling and Analysis*, 5.ª ed. New York, NY, EE. UU.: McGraw-Hill Education, 2014.
4. J. D. Hunter, “Matplotlib: A 2D Graphics Environment”, *Computing in Science & Engineering*, vol. 9, no. 3, pp. 90–95, 2007. Disponible en: `https://matplotlib.org`
5. C. R. Harris *et al.*, “Array programming with NumPy”, *Nature*, vol. 585, pp. 357–362, 2020. Disponible en: `https://numpy.org`

9. Anexos

Anexo A. Instalación y ejecución

Repositorio del proyecto: Repositorio GitHub

```
git clone https://github.com/NullDistortion/Repositorio-Simulacion.git
cd Repositorio-Simulacion/ape1
python3 -m venv venv
```

```
source venv/bin/activate
pip install --upgrade pip
pip install -r requirements.txt
python main.py
```

Anexo B. Declaración de uso de inteligencia artificial

Siguiendo los principios de transparencia y honestidad de la asignatura, declaro que para el desarrollo del código me apoyé en herramientas de inteligencia artificial generativa para organizar la arquitectura del programa, armar la vista gráfica en Matplotlib, preparar los archivos del repositorio y dar formato al documento en LaTeX

Prompt utilizado para el desarrollo del código:

> *"Partiendo del pseudocódigo implementa el modelo matemático de simulación atmosférica en Python utilizando una arquitectura Modelo-Vista-Controlador (MVC) con un diseño modular orientado a objetos, respetando las normas de Clean Code y los principios SOLID (especialmente el Principio de Responsabilidad Única para separar la lógica matemática, la visualización en Matplotlib y el control de flujo, así como el Principio Abierto/Cerrado para la parametrización de los pesos del modelo)."*

Declaración de Delegación en Inteligencia Artificial Generativa (Traducción GAIDeT):

Los autores declaran el uso de inteligencia artificial generativa en el proceso de investigación y redacción. De acuerdo con la taxonomía GAIDeT (2026), las siguientes tareas fueron delegadas a herramientas de IAG bajo supervisión humana total:

- Generación de ideas (*Idea generation*)
- Generación de código (*Code generation*)
- Optimización de código (*Code optimization*)
- Automatización de procesos (*Process automation*)
- Creación de algoritmos para análisis de datos (*Creation of algorithms for data analysis*)
- Validación (*Validation*)
- Limpieza de datos (*Data cleaning*)
- Análisis de datos (*Data analysis*)
- Visualización (*Visualization*)

La herramienta de IAG utilizada fue: **Gemini Argon Alto y NotebookLM**.
La responsabilidad del manuscrito final recae íntegramente en los autores.
Las herramientas de IAG no figuran como autores y no asumen responsabilidad por los resultados finales.
Declaración presentada por: Emilio Josue Flores Gallardo.